Machine Learning Foundations Group Assignment
Applied Machine Learning Project

Prof. Matteo Turilli

1 Description

Welcome to the Group Assignment! This project challenges you to move beyond theoretical exercises and apply machine learning techniques to a problem of your choosing, grounded in real-world data. It is a culminating project that brings together all key elements from the course: from thoughtful data preprocessing to rigorous model evaluation, from understanding the pitfalls of data leakage to interpreting model results within a broader context. You will work in teams, simulating how machine learning solutions are developed in collaborative environments.

Rather than focusing on model performance alone, this assignment emphasizes critical thinking: making informed methodological decisions, communicating insights effectively, and understanding the implications of your results. Whether your project succeeds in producing a high-performing model or reveals deeper complexities of the data, your ability to justify your approach and reflect on its impact will be central to your evaluation.

An essential lesson in this process is the value of negative results. In scientific and engineering practice, not every model will outperform a baseline, not every hypothesis will hold, and not every dataset will support a strong signal. These outcomes are not failures — they are findings, and how you analyze and present them will be a key aspect of your final evaluation. A project that identifies why a modeling approach didn't work, or which aspects of the data undermined generalization, can demonstrate just as much insight as one that delivers a high F1 score.

You are also expected to scope your project realistically within the computational and time constraints available to you. This includes being deliberate in model selection: a deep neural network might sound appealing, but may not be necessary—or feasible—for a relatively structured dataset. Use the strategies we've discussed in class to manage resources: subsampling, dimensionality reduction, and simpler baselines can be powerful tools when used wisely. Justifying these trade-offs is part of sound ML engineering.

Finally, project scoping and framing are critical. You should define a clear, measurable objective, understand your dataset's structure and limitations, and select evaluation metrics that align with the task. It is better to tightly frame a solvable problem than to cast a wide net that you cannot cover effectively in the time available.

Throughout this process, you are not being evaluated solely on performance metrics. Instead, you will be assessed on the strength of your reasoning, the clarity of your communication, and your ability to reflect on your work's technical and contextual nuances. This is your opportunity to showcase what machine learning can do and what it means to apply it responsibly and rigorously.

2 Objectives and Learning Outcomes

This assignment synthesizes the foundational elements of machine learning through a hands-on group project that mirrors professional data science practice. By completing this project, students will engage in the entire lifecycle of an ML workflow—from problem definition to final evaluation and presentation—while critically examining the choices made at each stage. The project will help you to:

- **Frame a real-world problem in ML terms**. Translate an open-ended, domain-specific challenge into a structured learning task, such as classification, regression, or ranking. You will learn how to define clear objectives, understand the business or societal context, and assess whether ML is the right tool for the job.
- **Develop data-centric thinking**. Perform principled exploratory data analysis (EDA), understand dataset limitations, recognize potential biases, and apply transformations appropriately. You will learn to prevent data leakage, handle missing and imbalanced data, and ensure that your models generalize meaningfully.
- **Build and justify ML pipelines**. Implement and evaluate end-to-end workflows using Scikit-learn, Pandas, and other standard libraries. You will experiment with multiple models and baseline strategies, document your decisions, and explain why certain approaches are more suitable given your data, resources, and goals.
- **Evaluate models rigorously**. Go beyond accuracy. Choose appropriate metrics (e.g., ROC-AUC, F1, MAE), apply cross-validation, and critically analyze overfitting, underfitting, and tradeoffs between precision and recall. You will gain experience with performance uncertainty, metric variance, and validation strategy design.
- **Scope solutions under constraints**. Make design decisions based on time, computing, and data availability limitations. You will learn to manage complexity, avoid over-engineering, and iterate efficiently. Practical constraints are not obstacles to success. Instead, they are part of the learning.

- **Handle and present negative results**. Embrace failed experiments as learning opportunities. Not all models will outperform a baseline, nor are all datasets equally learnable. You will be evaluated on your ability to diagnose, reflect on, and explain why certain approaches didn't work, not just describe those that did.
- **Collaborate effectively in technical teams**. Coordinate within your group using shared tools (e.g., Colab, GitHub, ReviewNB), communicate across roles (e.g., EDA lead, modeling lead), and maintain reproducibility and transparency throughout the development process.
- **Communicate findings**. Craft a concise report, a clear, engaging presentation, and a synthetic but impactful project poster. Show that you can summarize complex workflows, convey insights from data and model behavior, and explain your results and their implications with clarity and integrity.

3 Requirements

Your project must demonstrate competence in the full lifecycle of a machine learning pipeline. Each stage should be documented, justified, and reproducible. The sections below outline what must be included in your final deliverables, along with guidance on best practices and expectations.

3.1 Project Definition

This section sets the foundation of your project by requiring you to identify a real-world problem that can be framed as a machine learning task. You will select a suitable dataset, justify its relevance, and clearly define your learning objective (e.g., classification, regression, clustering or ranking). The strength of your project depends on this initial scoping: the problem should be appropriately challenging yet feasible within your team's computational and time constraints. This step also requires early communication with the instructor to validate your project direction.

- **Problem selection**:
  - Choose a real-world task that can be meaningfully addressed through supervised or unsupervised learning. Clearly state whether it is a classification, regression, or clustering problem.
- **Dataset selection**:
  - Your dataset must contain a sufficient number of records (ideally >10,000, but quality matters more than size).
  - It must be relevant to your problem and include structured features appropriate for modeling.
  - Public datasets are preferred (e.g., Kaggle, Hugging Face, UCI, DrivenData). Proprietary or private data may be used with prior approval.

3.2 Data Exploration and Preprocessing

Effective machine learning begins with understanding the data. This section focuses on exploring your dataset through visual and statistical methods, identifying issues such as missing values, outliers, and feature imbalances. You will apply transformations to prepare the data for modeling while explicitly avoiding data leakage. Emphasis is placed on documenting decisions and reasoning about how your data representations affect downstream learning. This stage is not about cleaning the data blindly, but about making your data usable, trustworthy, and informative.

- **Exploratory data analysis (EDA):**
  - Perform statistical summaries, correlations, and visualizations.
  - Identify potential data quality issues, outliers, feature distributions, and target imbalance.
- **Missing values:**
  - Identify missing values and justify your imputation strategy (e.g., mean, median, model-based).
- **Outliers:**
  - Analyze the impact of extreme values on modeling, and decide if to transform, clip, or retain them.
- **Feature engineering:**

  - Apply at least one transformation technique, such as encoding, normalization, discretization, text vectorization, or time-feature extraction.
  - Document and justify each transformation. Clearly distinguish between transformations fit on the training set and those applied to validation/test sets to avoid data leakage.

- **Data partitioning:**
  - Split your dataset into training, validation, and test sets.
  - Justify your split proportions based on data size and modeling goals.
  - For small datasets, apply k-fold cross-validation and report both the average and variance of performance.

3.3 Model Development

With a prepared dataset, you will design and implement several machine learning models that are suitable for your problem. The goal is not only to build performant models but to do so in a principled way. You will start with simple baselines and progress to more expressive models, ensuring that all implementations are wrapped in reproducible, modular pipelines. Attention should be given to code structure, clarity, and how preprocessing is integrated into the modeling workflow. The diversity of models and the clarity of your experimental design are key here.

- **Baselines:**
  - Include at least one trivial baseline (e.g., majority class predictor or zero-rule regressor).
  - Implement a classical ML model such as logistic regression, decision tree, or k-nearest neighbors as a realistic performance reference.
- **Advanced models:**
  - Include at least one higher-capacity model: e.g., random forest, gradient boosting, neural network, or support vector machine.
- **Pipelines:**
  - Build your ML pipeline (e.g., `Pipeline()` or equivalent) to encapsulate preprocessing and modeling.
  - Ensure the pipeline can be serialized and reused without manual steps.
- **Code organization:**
  - Clearly comment and structure your code.
  - Avoid running notebook cells out of order. Each section must be executable independently from top to bottom.

3.4 Model Evaluation and Selection

Here, you will rigorously assess your models using quantitative metrics and validation procedures appropriate to your task and dataset characteristics. This includes selecting metrics that reflect your project's goals (e.g., prioritizing recall over accuracy in imbalanced classification) and validating models via cross-validation or holdout sets. You will also systematically tune hyperparameters and compare models on raw scores, interpretability, robustness, and generalizability. A well-justified model selection is the outcome of this phase.

- **Evaluation metrics:**
  - Choose metrics appropriate to the task and class distribution.
  - Classification: Use precision, recall, F1-score, and ROC-AUC. Avoid reporting accuracy alone if classes are imbalanced.
  - Regression: Use RMSE, MAE, or R^2 as appropriate.
  - Clustering: Use silhouette score, Davies-Bouldin index, or adjusted mutual information.
  - Ranking: Use metrics such as precision@k, recall@k, mean average precision (MAP), or normalized discounted cumulative gain (nDCG), especially when the output must prioritize relevance or ordering over discrete classification.

- **Hyperparameter tuning:**
  - Use at least one systematic approach, such as `GridSearchCV`, `RandomizedSearchCV`, or `BayesSearchCV`.
  - Justify your hyperparameter ranges and your choice of tuning budget (e.g., number of combinations or iterations).
- **Model comparison:**
  - Present a summary table of model performance on validation and test sets.
  - Include both quantitative metrics and visualizations (e.g., confusion matrix, learning curves, ROC curve).
  - Interpret the results and clearly state which model is selected and why.

3.5 Interpretation and Reflection

This final section pushes you to think critically about your results. What insights can be drawn from the model's behavior? Why did certain approaches work better than others? You will analyze feature importance, failure cases, and potential biases in your data or modeling choices. Additionally, you will reflect on your group's process: how responsibilities were divided, how decisions were made, and what you learned technically and collaboratively. Emphasizing both successes and limitations is expected — honest reflection is part of rigorous ML practice.

- **Feature importance:**
  - Provide model interpretability using one or more of the following: feature importances, permutation importance, SHAP values, or LIME.
- **Failure modes:**
  - Analyze and explain at least one example where the model performs poorly.
  - Consider confusion matrix analysis, error distributions, or case studies of incorrect predictions.
- **Reflection on process:**
  - Discuss what worked, what didn't, and what surprised you.
  - Describe any ethical concerns, data limitations, or bias considerations.
  - Comment on group workflow: division of labor, tools used, and what you would change if you repeated the project.

4 Deliverables

Each group must submit the following components. Unless otherwise indicated, all deliverables must reflect the contributions of the entire group and be submitted via Blackboard by the deadline.

Depending on the size of your code and the computing requirements of your project, you should create a separate module or file(s) with your main classes and methods (or functions). Note that GitHub does not effectively handle conflict resolution in Jupyter notebooks, which complicates group collaboration. Use a Jupyter notebook as the 'interface' of your projects and load all your helper classes, methods, or functions at the beginning of your notebook. This way, you can collaborate on the code via GitHub and use the notebook solely to execute your pipeline and analysis. Finally, be aware that notebooks are slow! Depending on your computing requirements, you may need to run your pipeline as a stand-alone Python application and then load the results into a notebook for analysis and visualization.

1. **Final Pipeline, implemented in a Jupyter notebook and/or a stand-alone Python script/module. Deadline: 28/05**
   - A clean, readable, and fully executable code base composed of a notebook and/or stand-alone, properly commented Python code.
   - Code must include: data loading, preprocessing, modeling, evaluation, interpretation, and conclusions.
   - Cells must run top-to-bottom without errors. If used, stand-alone code must include deployment instructions and also execute without errors WARNING: Code that fails to compile will be deemed invalid and contribute 0 to the overall grade, regardless of how trivial the error may be. You must test the entire notebook/stand-alone code before submission.

- Use markdown cells for commentary and to explain rationale for modeling decisions. If you code standalone scripts/modules, properly comment your code.
- Ensure all data manipulations are reproducible and free of data leakage.
- If you need to code several functions/classes+methods, you may want to code them into a separate file(s) and load them from the notebook. This will help to keep your code clean and more easily testable, portable, and reproducible.

2. **Project Report. Deadline: 28/05**
   - **Format:**
     - Max 2000 words, excluding tables, diagrams, plots, and references.
     - Submit in PDF as a separate document.
   - **Content should include:**
     - Title, team names, and dataset used.
     - GitHub repository for the code. Note: the repository must include clear instructions for running the ML pipeline in the README.md file located in the root directory.
     - Problem statement and dataset summary.
     - Pipeline overview and modeling approach.
     - Key results, model comparison table, and metric interpretation.
     - Reflection on failures and limitations.
     - Summary of group collaboration and task division. WARNING: Every group member should commit their work to GitHub via regular commits. Failure to do so may lead to a different evaluation and grading of each group member.
   - Be concise and avoid duplicating notebook content verbatim.
   - Include references where appropriate (e.g., libraries, academic papers).
3. **Group Presentation (20 minutes + 5 min Q&A). Deadline: 21/05**
   - **Format:** Slide deck (PDF, PowerPoint, or Google Slides) with 10 slides.
   - Present the project in a clear narrative:
     - What was the problem, and why is it interesting?
     - What data did you use, and what challenges did you face?
     - What worked well, and what failed?
     - What did you learn, and how did the team work together?
   - All group members must speak during the presentation.
   - Practice timing: points will be deducted for overrunning.
4. **Poster Presentation (A1, single-page PDF). Deadline: 20/05**
   - **Format:** Academic-style research poster (landscape A1, 841mm x 594mm).
   - You may use PowerPoint, LaTeX (e.g., `beamerposter`), or any poster template tool.
   - **Sections must include:**
     - Title and Team Info: Project title, team members, affiliation (IE University).
     - Motivation and Problem Statement: Why this problem? Real-world impact?
     - Dataset Description: Source, size, preprocessing challenges.
     - Methodology: Pipeline diagram and brief description of models used.
     - Results: Summary table and selected visualizations (confusion matrix, ROC, feature importances).
     - Interpretation: Insights from model behavior, error analysis.
     - Reflection: What worked? What didn't? What did you learn?
     - References and Tools: Cite datasets, libraries, frameworks.
   - Design for clarity and visual communication. Avoid walls of text.
   - Poster session will be held in person, and the department will offer printing.

5 Evaluation Criteria

Each project will be graded out of 100 points, distributed across the following criteria. Use this rubric to guide your project planning, task allocation, and report writing. Strong projects are not just technically correct, but demonstrate critical reasoning, methodological care, and clear communication.

- **Data Exploration and Preprocessing (20 points)**
  - **18–20**: Comprehensive EDA with clear visualizations, thoughtful handling of missing values, outliers, and imbalances; no data leakage; excellent feature engineering with justification.
  - **14–17**: Adequate EDA and preprocessing, with minor gaps or partial justification; no critical errors.
  - $<$**14**: Poor or superficial analysis, unaddressed data issues, or leakage introduced in preprocessing.
- **Model Complexity and Justification (20 points)**
  - **18–20**: Strong rationale for model selection; includes both baselines and advanced models; models are well-matched to the problem and data constraints.
  - **14–17**: Models are relevant but lack clear justification or diversity; minor misalignments between problem and model type.
  - $<$**14**: Inadequate model variety or use of inappropriate algorithms for the task.
- **Evaluation Methodology and Rigor (20 points)**
  - **18–20**: Well-chosen metrics matched to the problem type and dataset; clear performance comparison; cross-validation or robust test strategy; interpretation of metric significance.
  - **14–17**: Some metrics are misaligned or weakly interpreted; evaluation is valid but incomplete.
  - $<$**14**: Overreliance on misleading metrics (e.g., accuracy for imbalanced classification); inadequate or absent test validation.
- **Code Quality and Reproducibility (10 points)**
  - **9–10**: Clear, well-organized, and commented code; uses pipelines; runs from top to bottom without errors.
  - **6–8**: Mostly clean code, some inconsistencies or minor execution issues.
  - $<$**6**: Poor code hygiene; missing documentation; broken pipeline logic.
- **Use of Advanced Methods (10 points)**
  - **9–10**: Use of advanced modeling (e.g., ensembles, neural networks) or interpretation (e.g., SHAP, LIME) justified and implemented correctly.
  - **6–8**: Advanced methods are attempted but not fully integrated or interpreted.
  - $<$**6**: No meaningful application of advanced methods.
- **Report Clarity and Structure (10 points)**
  - **9–10**: Clear, concise, and well-organized writing; covers all required elements; excellent use of tables and figures.
  - **6–8**: Adequate structure, minor issues in clarity or formatting.
  - $<$**6**: Disorganized or unclear; missing key sections or poorly supported claims.
- **Presentation Quality and Synthesis (10 points)**
  - **9–10**: Engaging and structured presentation; strong visual design; team participation; clear summary of problem, methods, and findings.
  - **6–8**: Adequate delivery; minor pacing or cohesion issues; some imbalance in speaker contributions.
  - $<$**6**: Poorly structured or hard to follow; disorganized visuals or unclear narrative.

Bonus: Exceptional originality, creativity, or impact in approach, dataset selection, or insight may be awarded up to +3 additional points at the instructor's discretion.

6 Group Work Ethic and Individual Accountability

This assignment is a collaborative effort that reflects real-world team-based ML engineering practice. By default, all group members will receive the same grade, provided there is clear evidence of equitable contribution and shared responsibility throughout the project. Effective collaboration includes dividing tasks fairly, supporting each other, integrating individual contributions into a coherent submission, and delivering the presentation and poster.

However, individual grades will be assigned based on each member's verifiable contributions in the presence of credible evidence that one or more team members did not meaningfully contribute, or that a subset of the group overwhelmingly carried the workload. These will be assessed using traceable records such as GitHub commit history, versioned Google Docs comments and edits, Colab access logs, and other timestamped digital evidence. Absence of such records will be interpreted as a lack of commitment and output.

In extreme cases of disproportionate contribution (e.g., if one or two members complete more than 70% of the workload), the group may be formally dissolved. Remaining members may be reassigned or required to submit a simplified individual report. Non-contributing members will be grouped and held independently responsible for a new, equivalent assignment with an adjusted deadline.

It is in everyone's interest to ensure a balanced distribution of work. If you encounter team dynamics or imbalanced effort issues, you should bring this to the professor's attention early, not after the deadline. The smart move—and the professional and ethical one—is to organize your team so that no such intervention is necessary.